

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ



МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

*Факультет Информационных технологий
Кафедра Информатики и информационных технологий*

направление подготовки

09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 7

Дисциплина: BackEnd-разработка

Тема: Работа с middleware в ASP.NET Core

Выполнил: студент группы 241 - 333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись _____
(Дата) (Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

Замечания: _____

Москва

2026

Цель работы

Освоить создание и подключение пользовательских middleware в ASP.NET Core, понять принцип работы конвейера обработки HTTP-запросов (порядок выполнения до/после `next()`), а также научиться использовать `HttpContext` для передачи данных между middleware и формирования ответов (заголовки, статус-коды, обработка ошибок).

Используемые технологии и инструментальные средства

В ходе выполнения работы использовались:

- ASP.NET Core Web API — платформа для разработки веб-приложений
- C# — язык программирования
- Visual Studio — среда разработки
- Middleware — механизм обработки HTTP-запросов
- `HttpContext` — объект контекста запроса

Ход выполнения работы

1. Создание базового Web API-приложения

На первом этапе был создан проект ASP.NET Core Web API без подключения Swagger. В файле `Program.cs` настроен запуск приложения и определены минимальные endpoint'ы.

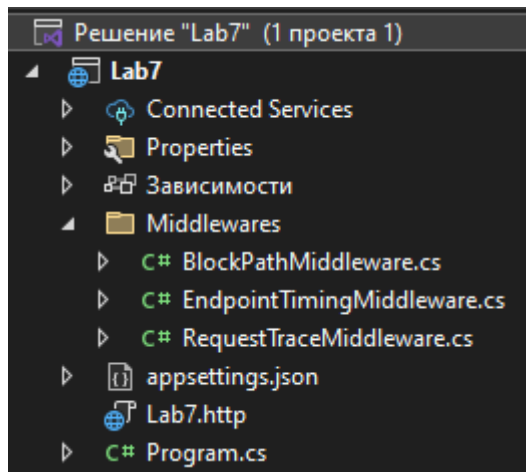


Рисунок 1. Структура проекта

2. Реализация middleware BlockPathMiddleware

Создан middleware, который блокирует доступ к определённым маршрутам. Логика работы:

- если путь начинается с /blocked, возвращается статус 403 Forbidden
- иначе запрос передаётся дальше по конвейеру

```
public class BlockPathMiddleware : IMiddleware
{
    Ссылка: 0
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        var path = context.Request.Path;
        if (path.StartsWithSegments("/blocked"))
        {
            context.Response.StatusCode = StatusCodes.Status403Forbidden;
            return;
        }
        else
        {
            await next(context);
        }
    }
}
```

Рисунок 2. Реализация BlockPathMiddleware

3. Реализация middleware RequestTraceMiddleware

Создан middleware для трассировки запросов. Функциональность:

- генерируется уникальный TraceId (Guid)
- добавляется заголовок X-Trace-Id
- значение сохраняется в HttpContext.Items
- выполняется следующий middleware

```

public class RequestTraceMiddleware : IMiddleware
{
    Ссылка: 0
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        var traceId = Guid.NewGuid().ToString();

        context.Response.Headers.Append("X-Trace-Id", traceId);
        context.Items["TraceId"] = traceId;

        await next(context);
    }
}

```

Рисунок 3. Реализация RequestTraceMiddleware

4. Реализация middleware EndpointTimingMiddleware

Создан middleware для измерения времени выполнения запроса. Логика:

- используется Stopwatch
- измерение начинается до вызова next()
- после выполнения endpoint добавляется заголовок X-Endpoint-Elapsed-Ms.

```

public class EndpointTimingMiddleware : IMiddleware
{
    Ссылка: 0
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        var sw = Stopwatch.StartNew();

        await next(context);

        sw.Stop();
        context.Response.Headers.Append("X-Endpoint-Elapsed-Ms",
            ((long)sw.Elapsed.TotalMilliseconds).ToString());
    }
}

```

Рисунок 4. Реализация EndpointTimingMiddleware

5. Подключение middleware

Middleware были подключены в строго заданном порядке, так как порядок подключения middleware напрямую влияет на обработку HTTP-запроса. Каждый компонент конвейера выполняется последовательно до вызова

next(), а затем в обратном порядке после его завершения. Таким образом, middleware образуют цепочку обработки, где:

- код до await next() выполняется при входе запроса,
- код после — при формировании ответа.

Неправильный порядок может привести к некорректной работе приложения (например, заголовки не будут добавлены или запрос будет обработан не полностью).

```
app.UseMiddleware<BlockPathMiddleware>();  
app.UseMiddleware<RequestTraceMiddleware>();  
app.UseMiddleware<EndpointTimingMiddleware>();
```

Рисунок 5. Подключение Middleware

6. Реализация endpoint'ов

Были реализованы три минимальных endpoint'а:

1. GET /ping, возвращающий строку pong.
2. GET /trace, возвращающий TraceId, сохранённый в HttpContext.Items.
3. GET /error, выбрасывающий исключение для тестирования обработки ошибок.

```
app.MapGet("/ping", () =>  
{  
    return "pong";  
});  
  
app.MapGet("/trace", (HttpContext context) =>  
{  
    return Results.Ok(context.Items["TraceId"]);  
});  
  
app.MapGet("/error", () =>  
{  
    throw new Exception("Error for handling");  
});
```

Рисунок 6. Реализация endpoint'ов

7. Результаты тестирования

URL	Метод	Результат
/ping	GET	pong
/trace	GET	TraceId
/blocked/test	GET	403 Forbidden
/error	GET	500 Internal Server Error

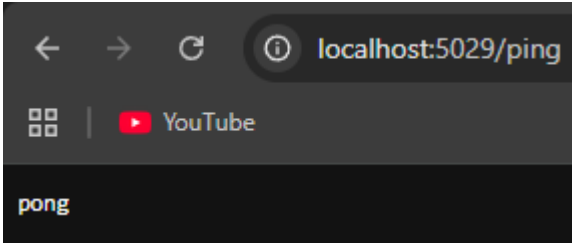


Рисунок 7. Результат /ping

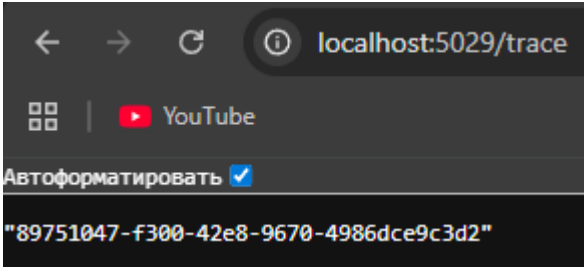


Рисунок 8. Результат /trace

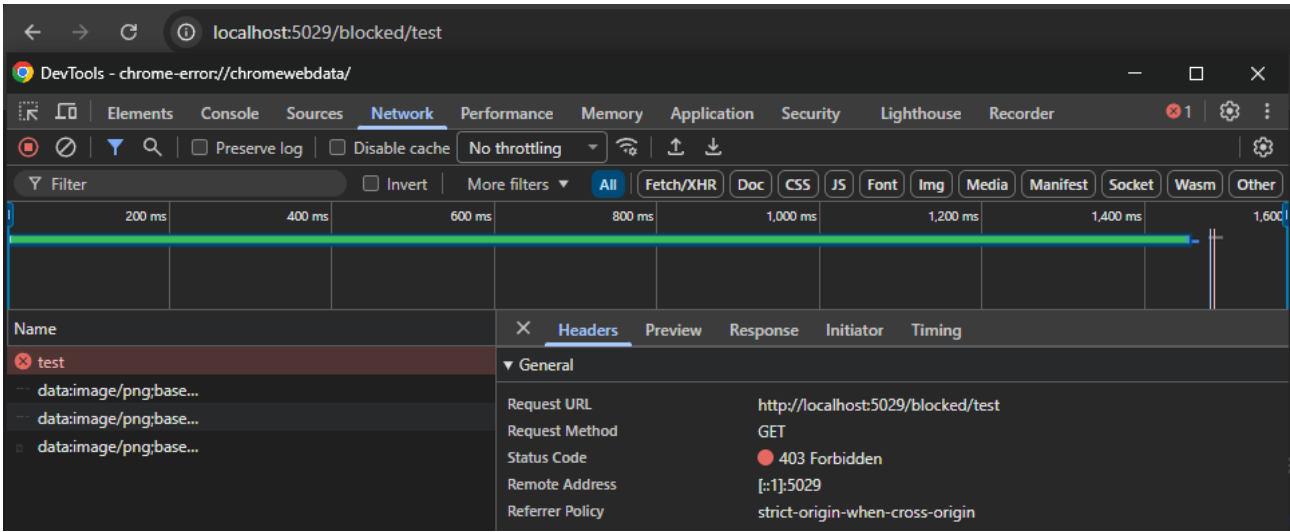


Рисунок 9. Результат /blocked/test

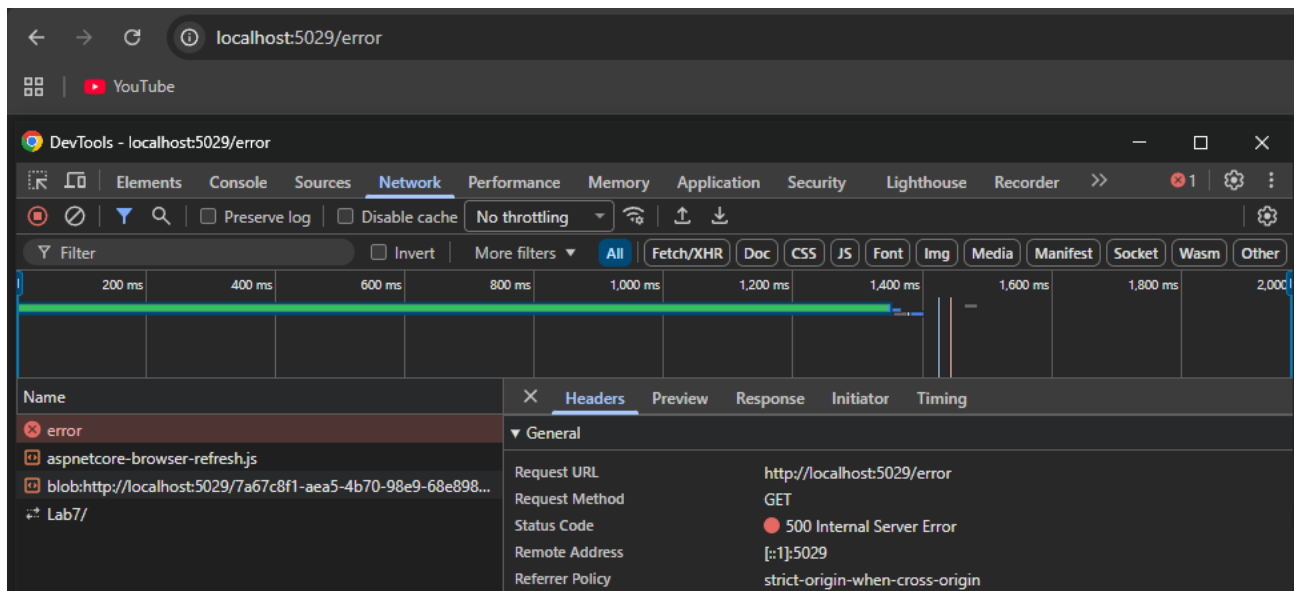


Рисунок 10. Результат /error

8. Возможные сложности

В процессе выполнения могли возникнуть следующие сложности:

- Понимание принципа работы pipeline
- Осознание важности порядка middleware
- Работа с HttpContext.Items
- Различие между IMiddleware и обычным middleware
- Передача данных между middleware

Вывод

В ходе выполнения лабораторной работы были изучены принципы создания и использования пользовательских middleware в ASP.NET Core, а также особенности работы конвейера обработки HTTP-запросов. Были освоены механизмы передачи данных между компонентами через HttpContext, добавления и управления заголовками HTTP-ответа, а также обработки маршрутов и возникающих ошибок. Практическая реализация позволила наглядно понять порядок выполнения middleware до и после вызова следующего компонента, что дало более глубокое представление о внутренней архитектуре ASP.NET Core и принципах обработки запросов в веб-приложениях.

Листинг кода

Листинг 1. Файл Program.cs:

```
using Lab7.Middlewares;

var builder = WebApplication.CreateBuilder(args);

// Регистрация middleware в DI-контейнере
builder.Services.AddTransient<BlockPathMiddleware>();
builder.Services.AddTransient<RequestTraceMiddleware>();
builder.Services.AddTransient<EndpointTimingMiddleware>();

var app = builder.Build();

// Подключение middleware в правильном порядке
app.UseMiddleware<BlockPathMiddleware>();
app.UseMiddleware<RequestTraceMiddleware>();
app.UseMiddleware<EndpointTimingMiddleware>();

// Endpoint для проверки работы
app.MapGet("/ping", () => "pong");

// Возврат TraceId из HttpContext
app.MapGet("/trace", (HttpContext context) =>
{
    return Results.Ok(context.Items["TraceId"]);
});

// Генерация ошибки
app.MapGet("/error", () =>
{
    throw new Exception("Error for handling");
});

app.Run();
```

Листинг 2. Файл BlockPathMiddleware.cs:

```
namespace Lab7.Middlewares
{
    public class BlockPathMiddleware : IMiddleware
    {
        public async Task InvokeAsync(HttpContext context, RequestDelegate next)
        {
            var path = context.Request.Path;

            // Проверка пути запроса
            if (path.StartsWithSegments("/blocked"))
            {
                // Возврат 403 без передачи дальше
                context.Response.StatusCode = StatusCodes.Status403Forbidden;
                return;
            }
            else
            {
                // Передача запроса дальше
                await next(context);
            }
        }
    }
}
```

Листинг 3. Файл RequestTraceMiddleware.cs:

```
namespace Lab7.Middlewares
{
    public class RequestTraceMiddleware : IMiddleware
    {
        public async Task InvokeAsync(HttpContext context, RequestDelegate next)
        {
            // Генерация уникального идентификатора запроса
            var traceId = Guid.NewGuid().ToString();

            // Добавление в заголовки ответа
            context.Response.Headers.Append("X-Trace-Id", traceId);

            // Сохранение в HttpContext
            context.Items["TraceId"] = traceId;

            await next(context);
        }
    }
}
```

Листинг 4. Файл EndpointTimingMiddleware.cs:

```
using System.Diagnostics;
namespace Lab7.Middlewares
{
    public class EndpointTimingMiddleware : IMiddleware
    {
        public async Task InvokeAsync(HttpContext context, RequestDelegate next)
        {
            var sw = Stopwatch.StartNew();

            // Добавление времени выполнения в заголовок
            context.Response.OnStarting(() =>
            {
                sw.Stop();
                context.Response.Headers.Append("X-Endpoint-Elapsed-Ms",
                    ((long)sw.Elapsed.TotalMilliseconds).ToString());

                return Task.CompletedTask;
            });

            // Выполнение следующего middleware / endpoint
            await next(context);
        }
    }
}
```